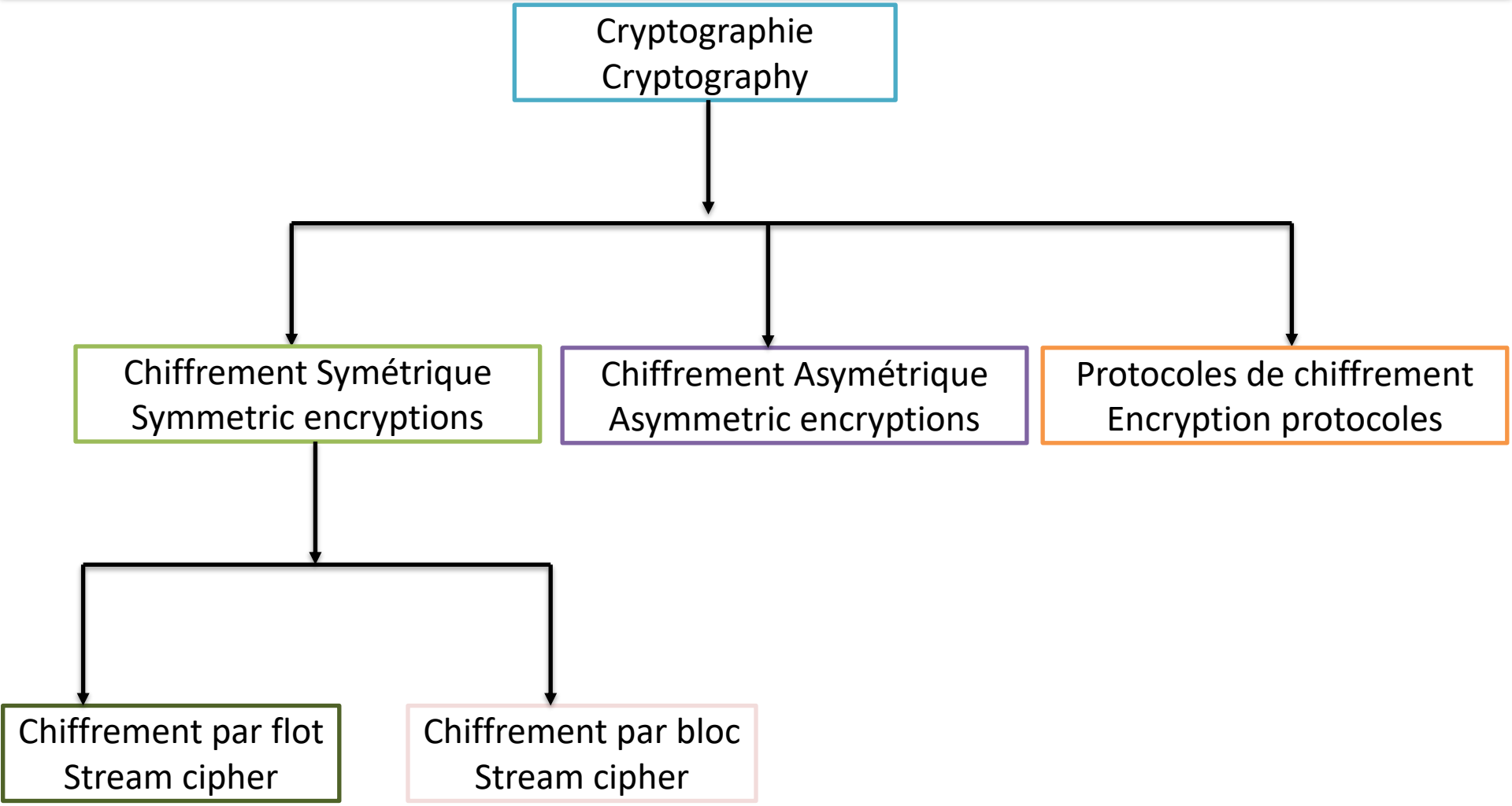
Cryptographie
Chapitre 1
Chiffrement par flot ou flux
Steam cipher

Présenté par Benaribi

Plan du cours

- Rappel
- Introduction au chiffrement par flot
- One time pad ou Masque jetable, Vernam
- Générateurs de nombres
- Chiffrement à flot à base de registre de décalage de registre
  - Principe Linear feedback shift registers LFSRs
  - Attaque LFSRs
  - Alternatives à LFSRs

Introduction



Cryptographie à clé secrète

- Un schéma de chiffrement à clé secrète est défini par un espace des messages et 3 algorithmes :

Un schéma de chiffrement à clé secrète est défini par 3 algorithmes :

$\mathrm{Gen}(1^n) = k$ algo probabiliste de génération de clé

$\mathrm{Enc}_k(m) = c$ algo probabiliste de chiffrement

$\mathrm{Dec}_k(c) = m$ algo déterministe de déchiffrement

$$\forall k = Gen(1^n) \quad \forall m \in \mathscr{M} \quad \mathrm{Dec}_k(\mathrm{Enc}_k(m)) = m$$

$$\forall n, \forall k = Gen(1^n) \quad |k| \geqslant n$$

Cryptographie à clé secrète

rappel Secret parfait (formel)

- Le schéma de chiffrement (Gen, Enc, Dec) **assure le secret parfait** si quelque soit la distribution de probabilité sur l'espace des messages, quelque soit le message m et quelque soit le message chiffré c de probabilité non nulle, on a

$$P(M = m \mid C = c) = P(M = m)$$

Formulation « expérimentale »

- Un adversaire A choisit deux messages m0 et m1 quelconques.
- L'un de ces deux messages est choisi uniformément et chiffré en c.
- A doit alors deviner si le message chiffré c correspond à m0 ou à m1.
- Un schéma est **parfaitement indistinguable** si A ne peut pas deviner la bonne réponse avec un probabilité supérieure à 1/2.

Équivalence et limitation

- Un schéma de chiffrement assure le secret parfait si et seulement si il est parfaitement indistinguable.
- **Théorème** Si (Gen, Enc, Dec) assure le secret parfait alors l'espace des clés est au moins aussi grand que l'espace des messages.
- **Principes de Kerckhoffs** Un système cryptographique doit pouvoir tomber entre les mains de l'ennemi : la sécurité doit reposer uniquement sur la clef, on doit supposer l'algorithme connu.

==> Donc d'après les principes de Kerckhoffs que la sécurité d'un algorithme de chiffrement symétrique par flot repose sur la qualité du générateur de clé .

Sécurité calculatoire

Configurations :

A. Un ordinateur de bureau avec 4 coeurs à 2,5 GHz : 2^{36} FLOPS

B. Cluster du laboratoire de maths : 2^{40} FLOPS

C. Supercomputer (à 133 million de dollars) : 2^{50} FLOPS

Nbre opérations		Config. A	Config. B	Config. C
Clé W.E.P. :	2^{40}	16 sec.	1 sec.	1 μsec.
Clé D.E.S. :	2^{56}	12 jrs	18 h	1 mn.
Collision MD5 :	2^{64}	$8\,{}^{1}/_{2}$ ans	194 jrs	4 h.
perm. de lettres :	2^{88}	142000 mill.	9000 mill.	8 mill.
Hill de dim. 5 :	2^{115}	10^{6} univers	10^{5} univers	83 univers
Clé 128 bits :	2^{128}	10^{10} univers	10^{8} univers	10^{5} univers
Clé 192 bits :	2^{192}	10^{11} univers^2	10^{10} univers^2	10^{7} univers^2

jusqu'à 2021, en février 2021, le supercalculateur le plus puissant au monde était Fugaku, installé à Kobe, au Japon. Fugaku a une puissance de calcul de 442 petaflops, ce qui en fait le plus puissant supercalculateur au monde.

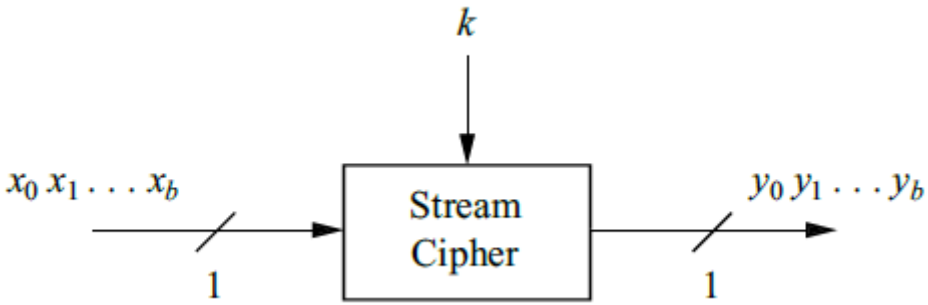
Sécurité Sémantique

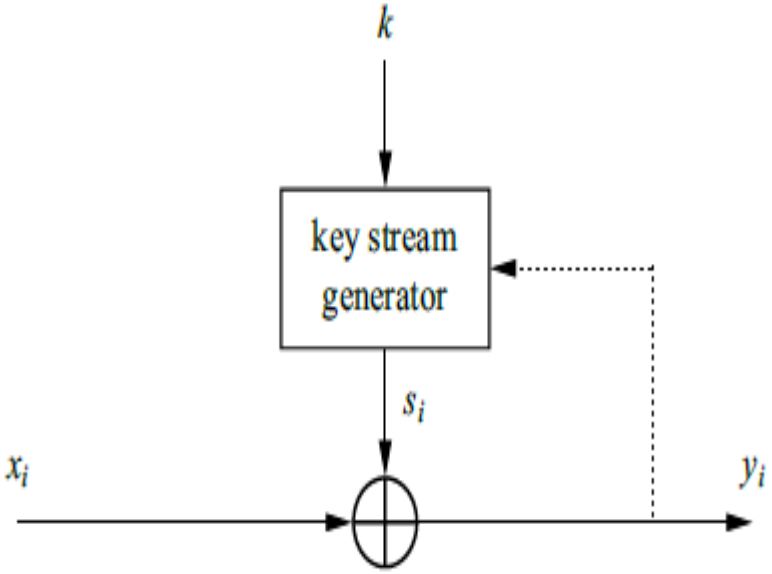
La sécurité sémantique stipule qu'un attaquant ne peut tirer aucune information du chiffrement même s'il connaît un ensemble fini de textes clairs M. La sécurité sémantique dans la cryptographie fait référence à la protection de la signification ou du contenu d'un message crypté. Cela signifie empêcher les attaquants d'accéder à la signification d'un message crypté.

- Un schéma est (t,ε)-sécurisé si un adversaire disposant d'un temps de calcul au plus t peut casser le chiffrement avec probabilité au plus ε
- Dans ce contexte, le temps de calcul (t) et la probabilité (ε) sont utilisés pour mesurer la sécurité d'un schéma de chiffrement. Plus t est grand et ε est petit, plus le chiffrement est considéré comme sécurisé..

Chiffrement par flot

- Les chiffrements par flots chiffrent les bits individuellement. Cela est accompli en ajoutant un bit d'un flot de clé à un bit de texte en clair. Il existe des chiffrements par flots synchrones où le flot de clé dépend uniquement de la clé, et des asynchrones où le flot de clé dépend également du texte chiffré.



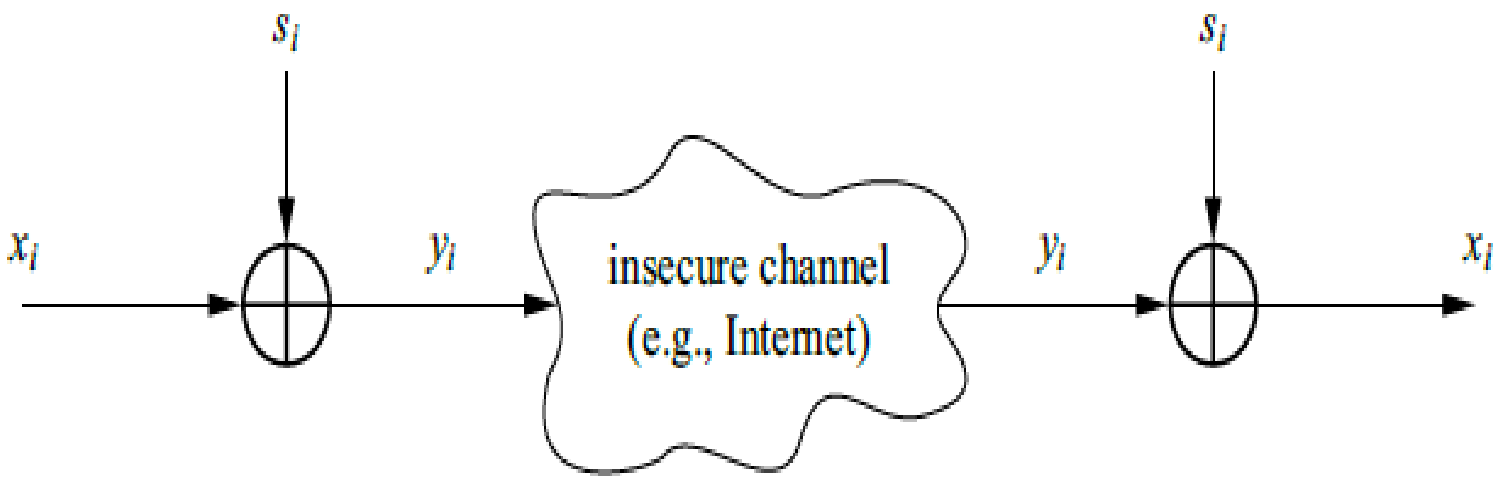


Chiffrement par flot

Definition le chiffrement et le déchiffrement est réalisé bit à bit soit $x_i, y_i, s_i \in \{0,1\}$.

Chiffrement : $yi = E_{si}(x_i) \equiv x_i + s_i \bmod 2$

Déchiffrement : $xi = D_{si}(yi) \equiv yi + s_i \bmod 2$



Chiffrement par flot

Questions :

1. **Pourquoi la fonction de chiffrement et déchiffrement sont les mêmes.?**
2. **Pourquoi utiliser modulo 2 dans le chiffrement.?**
3. **Comment générer les flots (flux de clés) S_i.?**

Réponses : Q1 la même fonction

- La fonction de chiffrement est : $E_{si}(xi) \equiv yi \equiv xi + si \bmod 2$.
- La fonction de déchiffrement est : $D_{si}(yi) \equiv yi + si \bmod 2$
- Si je remplace yi dans la fonction de chiffrement ça dans la fonction de déchifffrement ça donne :

$$
\begin{aligned}
D_{si}(yi) &\equiv yi + si \bmod 2 \\
&\equiv (xi + si) + si \bmod 2 \\
&\equiv xi + si + si \bmod 2 \\
&\equiv xi + 2si \bmod 2 \\
&\equiv xi + 0 \bmod 2 \\
&\equiv xi \bmod 2
\end{aligned}
$$

Chiffrement par flot

Réponse :Pourquoi utiliser modulo 2 dans le chiffrement.?

x_i	s_i	$y_i \equiv x_i + s_i \bmod 2$
0	0	0
0	1	1
1	0	1
1	1	0

Correspond au XOR →

x_i	s_i	y_i
0	0	0
0	1	1
1	0	1
1	1	0

Il a une ½ que ça soit un zéro ou un 1. offre une balance parfaite dans le système binaire.

Chiffrement par flot

Réponse :Pourquoi utiliser modulo 2 dans le chiffrement.?

x_i	s_i	$y_i \equiv x_i + s_i \bmod 2$
0	0	0
0	1	1
1	0	1
1	1	0

Correspond au XOR →

x_i	s_i	y_i
0	0	0
0	1	1
1	0	1
1	1	0

Il a une ½ que ça soit un zéro ou un 1. offre une balance parfaite dans le système binaire.

Chiffrement par flot

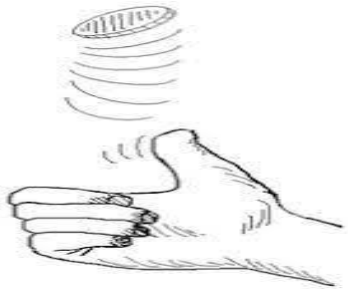
Réponse :Comment générer les flots (flux de clés) S_i.?

La génération de la suite des bits de la clé peut se faire

Par un générateur aléatoire
True random

Par un générateur pseudo aléatoire à l'aide de logiciel
Pseudo random

Par un générateur cryptographique pseudo aléatoire
Cryptographically pseudo randomo generator



Masque jetable, Vernam (One time pad)

- Gilbert Vernam en 1917 le masque jetable. Il est considéré comme un crypto-système parfaitement sûr . Il aurait été utilisé pour le téléphone rouge entre Washington et Moscou pendant la Guerre froide.
- La clé K est une suite aléatoire de nombres de Z/nZ (n=26 si on pense aux exemples déjà vus, mais on utilise habituellement le code ASCII à 256 caractères et on choisira alors n=256) de même longueur que le texte.
- Informations théorique est dit sécurisé càd (unbreakable), si:

✓ La clé est obtenue de façon aléatoire.

✓ **La clé ne doit pas être réutilisée.**

➢ plaintext1 ⊕ key = ciphertext1

➢ plaintext2 ⊕ key = ciphertext2

➢ plaintext1⊕ ciphertext1 = key

➢ Key ⊕ ciphertext1 = plaintext1

Il n'est pas utilisé à cause de la grande taille des clés, il est aussi Difficile de l'utiliser dans la réalité.

Des nombres aléatoires pour un chiffrement par flot incassable

Générateur de vrais hombre aléatoires TRNGs(True Random Number Generator)

- Les générateurs de vrais nombres aléatoires (TRNGs) se caractérisent par le fait que leur sortie ne peut être reproduite. Par exemple, si nous jouons 100 fois à pile ou face et que nous enregistrons la séquence de 100 bits qui en résulte, il sera pratiquement impossible pour quiconque sur Terre de reproduire le résultat.
- Les TRNGs sont basés sur des processus physiques. Exemples de pièces de monnaie, le lancer de dés, le bruit des semi-conducteurs, la variation d'un signal des horloges dans les circuits numériques etc….

Des nombres aléatoires pour un chiffrement par flot incassable

Les générateurs de nombres pseudo-aléatoires (PRNG)

- Les générateurs de nombres pseudo-aléatoires (PRNG) génèrent des séquences calculées à partir d'une valeur de départ. à partir d'une valeur initiale. Ils sont souvent calculés de manière récursive de la manière suivante :

$$s0 = \text{seed} \qquad \text{/*(la graine de départ)*/}$$

$$s_{i+1} = f(s_i),\ i = 0,1,\ldots N$$

Les générateurs congruentiels (linear congruential generator):

$s_0 = \text{seed}$

$s_{i+1} \equiv a\, s_i + b \bmod m,\ i = 0,1,\ldots N$ où a, b, m sont des constantes entières.

Exemple:

$s_0 = 12345$

$s_{i+1} \equiv 1103515245 s_i + 12345 \bmod 2^{31}$

Les générateurs de nombres pseudo-aléatoires (PRNG)

Les générateurs congruentiels (linear congruential generator):

- Une telle séquence est toujours périodique. il faudra choisir *a, b,m* de façon à ce que la période soit suffisamment grande. Par exemple, pour $m = 10$, $x_0 = a = b = 7$, la période est 4, et ce générateur n'est pas du tout satisfaisant. La période maximale est ´évidemment *m*.

- On choisit habituellement pour *m* le plus grand nombre premier descriptible par une machine

Des nombres aléatoires pour un chiffrement par flot incassable

Les générateurs de nombres pseudo-aléatoires cryptographique sécurisés (CSPRNG)

- Un CSPRNG est PRNG qui est imprévisible. De façon informelle, cela signifie qu'étant donné n bits de sortie du flux de clés s_i, s_{i+1}, . . . , si+n-1, où n est un nombre entier, il est infaisable du point de vue du calcul de prédire les bits suivants s_{i+n}, s_{i+n+1}, ...
- Une définition plus exacte est la suivante: étant donné n bits consécutifs du flux de clés, il n'y a pas d'algorithme en temps polynomial qui puisse prédire le bit suivant s_{n+1}.
- Exemple Blum-Blum-Shub générateur :

Les générateurs de nombres pseudo-aléatoires cryptographique sécurisés (CSPRNG)

En 1984, Blum et Goldwasser proposent le système de chiffrement suivant.

BLUM-GOLDWASSER : GÉNÉRATION DE CLEFS

1. Choisir aléatoirement deux grands nombres premiers p et q congrus à 3 modulo 4.
2. Calculer n = pq.
3. Calculer a et b les coefficients de Bezout de p et q, i.e. ap + bq = 1.
4. Choisir aléatoirement un seed X_0 / pgcd $(X_0,n)=1$.
5. Pour tout i = 1, . . . , t : calculer $x_i = x^2_{i-1} \bmod n$

Exemple p=11, q=19, n=209, X_0 =100.

100 = 1 × 100 et 209 = 1 × 209

Méthode d'Euclide :

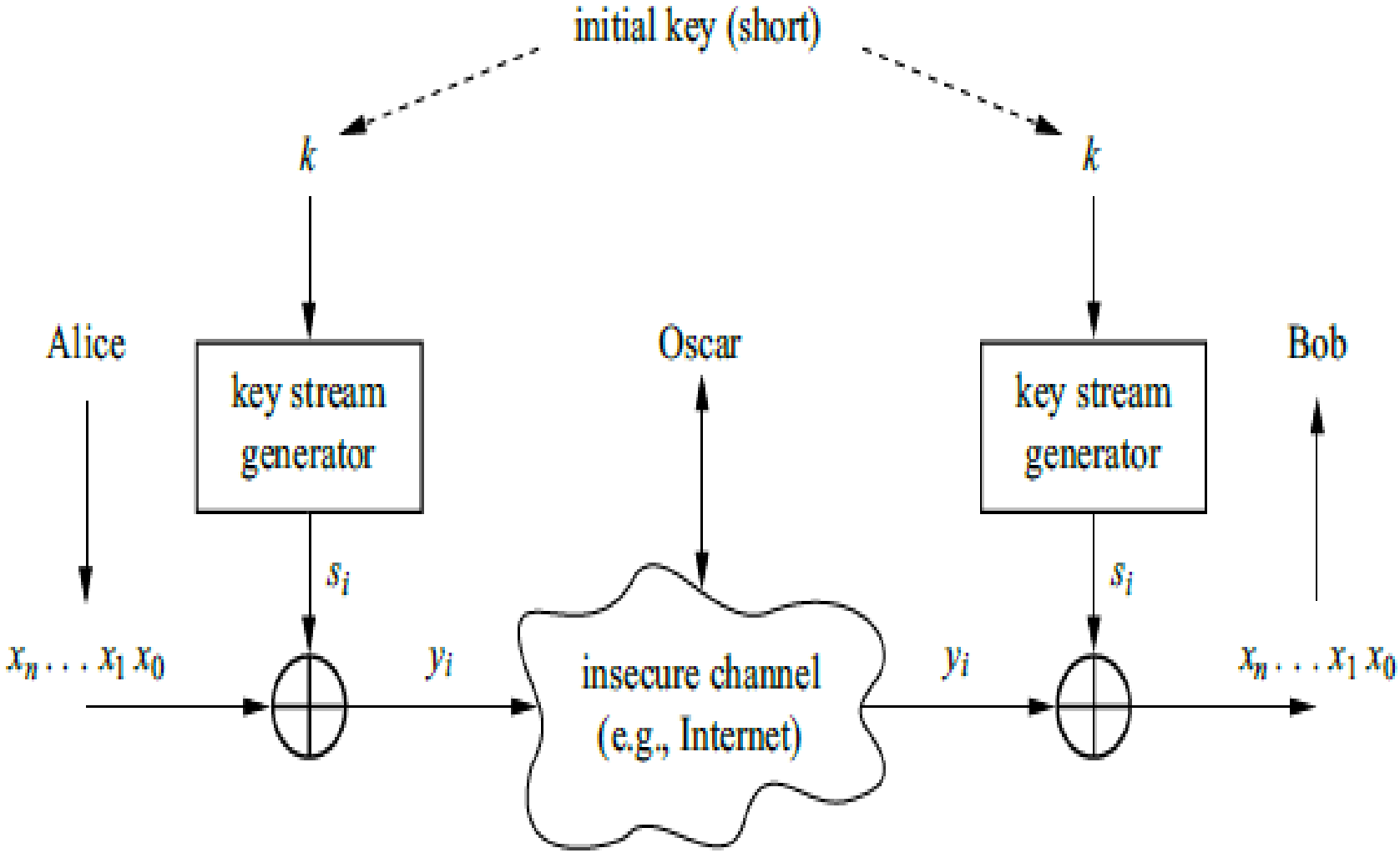
209 = 2×100 + 9

100 = 11×9 + 1

9 = 9×1 + 0Le PGCD est égal au dernier reste non nul : 1

$X_1 = 100^2 \bmod 209$

Chiffrement par flot



Chiffrement par flot avec un PRNG?

- De nombreux PRNG possèdent de bonnes propriétés statistiques, qui sont nécessaires pour un chiffrement de flot efficace.
- Si nous appliquons des tests statistiques à la séquence du flot de clés, la sortie devrait à peu près se comporter comme la séquence de bits générée par le lancer d'une pièce de monnaie.
- Il est donc tentant de supposer qu'un PRNG peut être utilisé pour générer le flot de clés. Mais tout cela n'est pas suffisant pour un chiffrement par flot puisque notre adversaire, Oscar, est intelligent. Considérons l'attaque suivante :

Chiffrement par flot avec un PRNG?

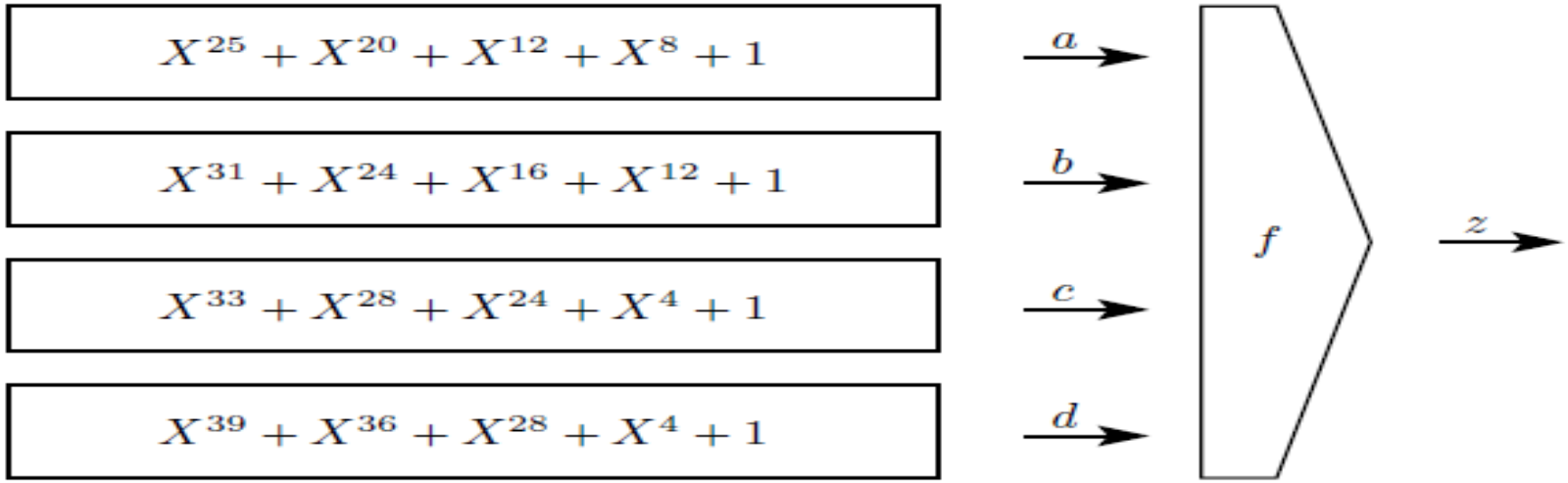
- Considérons l'attaque suivante :
- $S0$ = seed
- $Si+1 \equiv ASi+B \bmod m$, $i = 0,1, \ldots$ où nous choisissons m comme étant de longueur de 100 bits et Si,A,B∈{0,1, . . ,m-1}.
- La clé secrète comprend les valeurs (A,B) et éventuellement la graine S0, chacune ayant une longueur de 100. ce qui nous donne une clé de 200 bit. C'est plus que suffisant pour ce protéger d'une attaque par force brute.
- $yi \equiv xi + si \bmod 2$
- Supposons qu'un attaquant connaisse les 300 premiers bits du texte en clair Puisqu'il connaît certainement le texte chiffré, il peut maintenant calculer les 300 premiers bits du flux de clés comme : $si \equiv yi + xi \bmod m$, $i = 1,2, \ldots ,300$.
- PRNG: $S1 = (s1, \ldots , s100)$, $S2 = (s101, \ldots , s200)$ and $S3 = (s201, \ldots , s300)$. Oscar peut générer deux equations :

$$S2 \equiv AS1+B \bmod m$$
$$S3 \equiv AS2+B \bmod m$$

$$\longrightarrow \begin{cases} A \equiv (S2-S3)/(S1-S2) \bmod m \\ B \equiv S2-S1(S2-S3)/(S1-S2) \bmod m \end{cases}$$

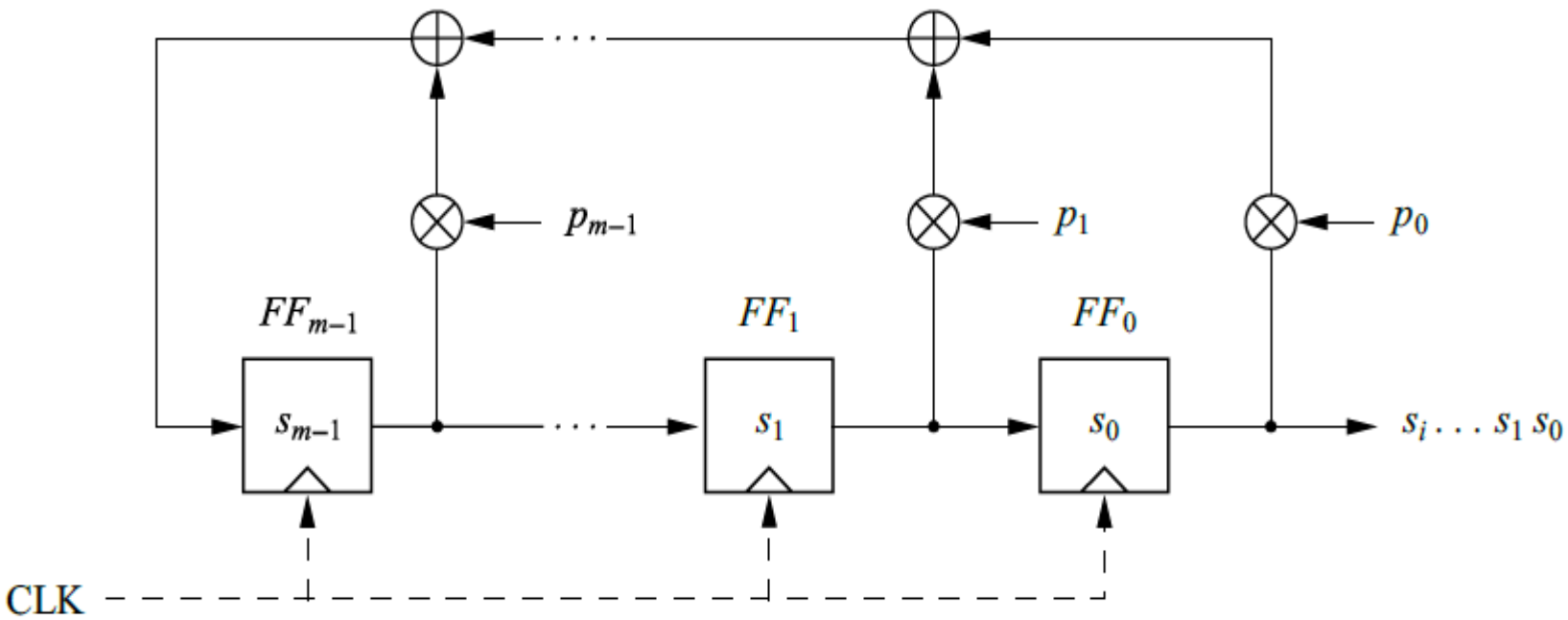
Chiffrement par flot à base de décalage de registre

- Une façon efficace de réaliser de longues séquences pseudo-aléatoires est d'utiliser des registres à décalage à rétroaction linéaire LFSR (Linear Feedback Shift Registers).
- même si un LFSR ordinaire produit une séquence avec de bonnes propriétés statistiques, il est d'un point de vue CPRNG faible. Cependant, des combinaisons de LFSR, telles que A5/4 (utilisé dans le GSM) ou le chiffrement Trivium et RC4 (utilisé dans SSL et WEP) peuvent produire des une suite sécurisés de bits.
- Exemple : S´sécurisation du protocole *Bluetooth*



Chiffrement par flot à base de décalage de registre: LFSR

- Un registre à décalage à rétroaction linéaire (ou LFSR, pour Linear Feedback Shift Regis- ter en anglais) est un circuit électronique ou une fonction logicielle qui permet de générer une suite récurrente linéaire donnée. Dans le cadre de ce sujet, nous nous restreindrons exclusivement au cas des suites récurrentes linéaires définies sur le corps des entiers modulo 2 : Z/2Z.



Chiffrement par flot à base de décalage de registre: LFSR

La sortie du LFSR est décrite par une équation récursive :

$$S_{i+3} = S_{i+1} + S_i \bmod 2$$

Avec les coefficients de retour (feedback coefficient) p_i

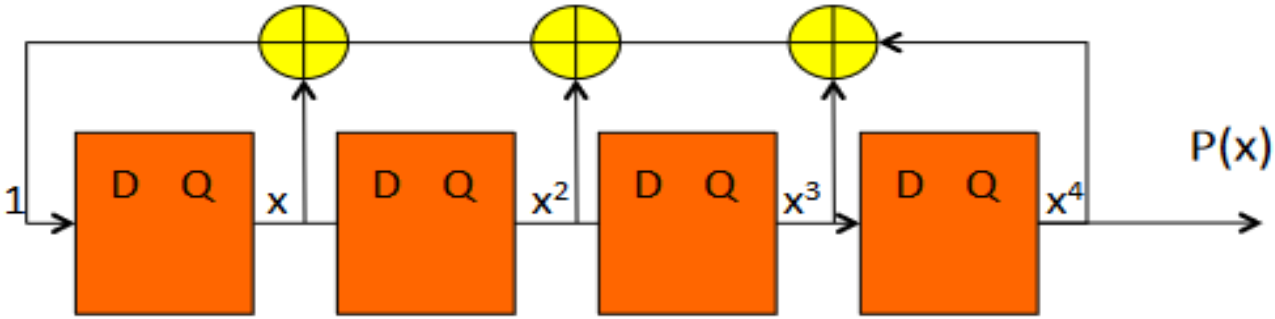
$$s_{i+m} \equiv \sum_{j=0}^{m-1} p_j \cdot s_{i+j} \bmod 2; \quad s_i, p_j \in \{0,1\};\ i = 0,1,2,\ldots$$

Les LFSRs sont typiquement décrits par des polynômes :

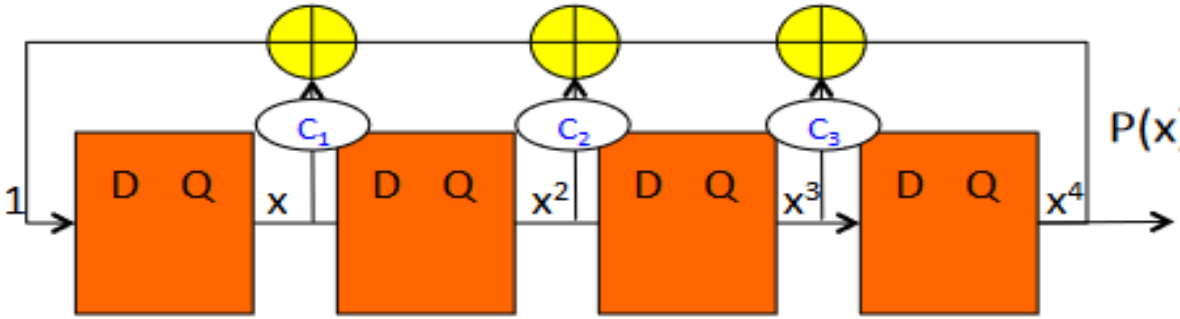
$$P(x) = x^m + p_{m-1}\, x^{m-1} + \ldots + p_1 x + p_0$$

Chiffrement par flot à base de décalage de registre: LFSR

LFSR



$P(x) = X^4+X^3+X^2+X+1$

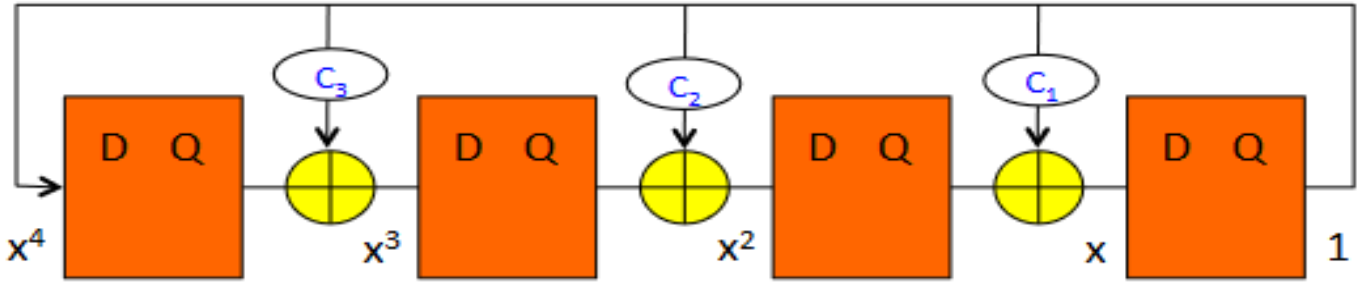


$P(x) = X^4+C_3X^3+C_2X^2+C_1X+1$

C_i=1 la connexion existe

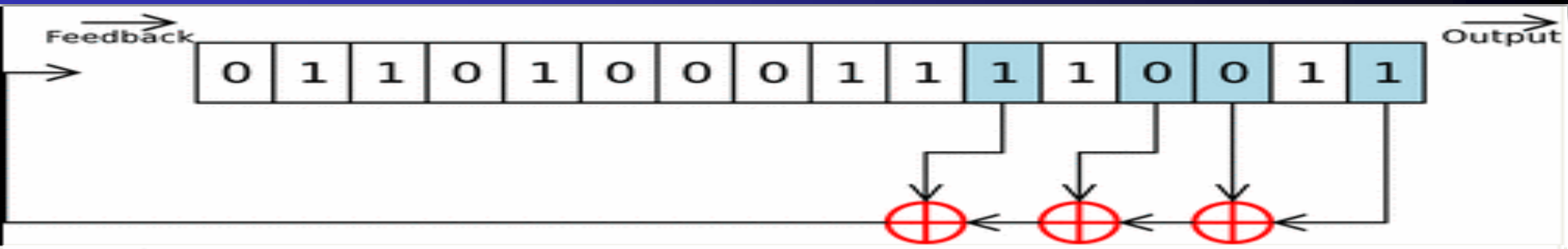
C_i=0 la connexion n'existe pas

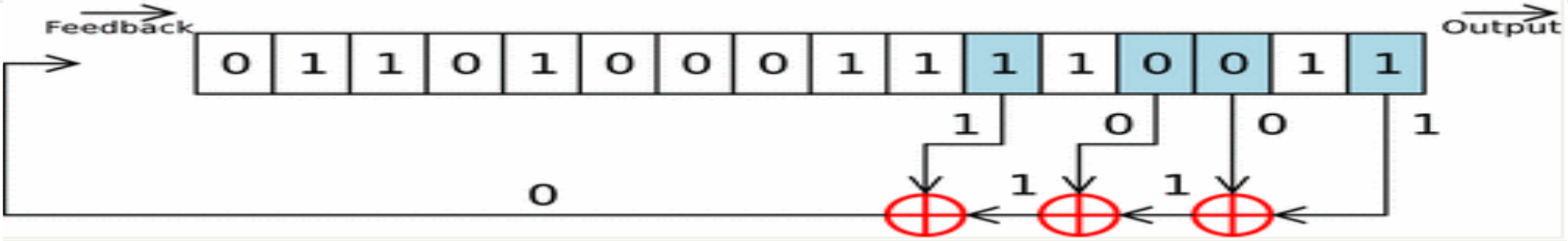
LFSR de Fibonacci

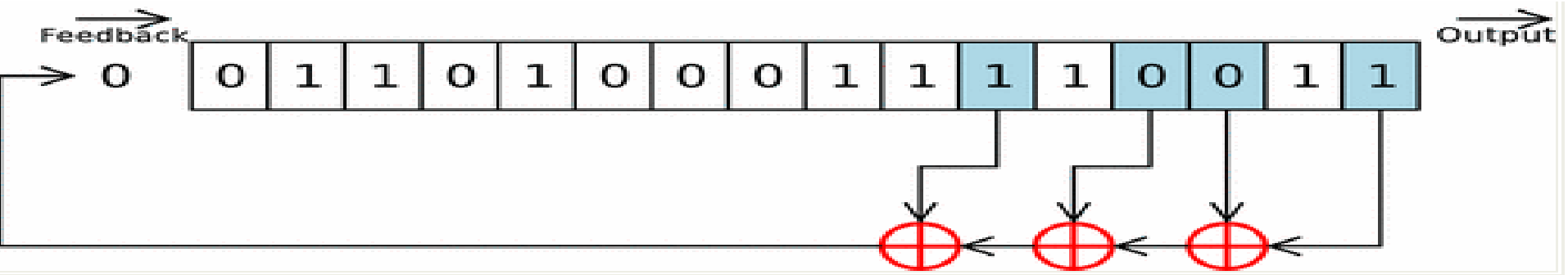


LFSR de Gallois équivalent

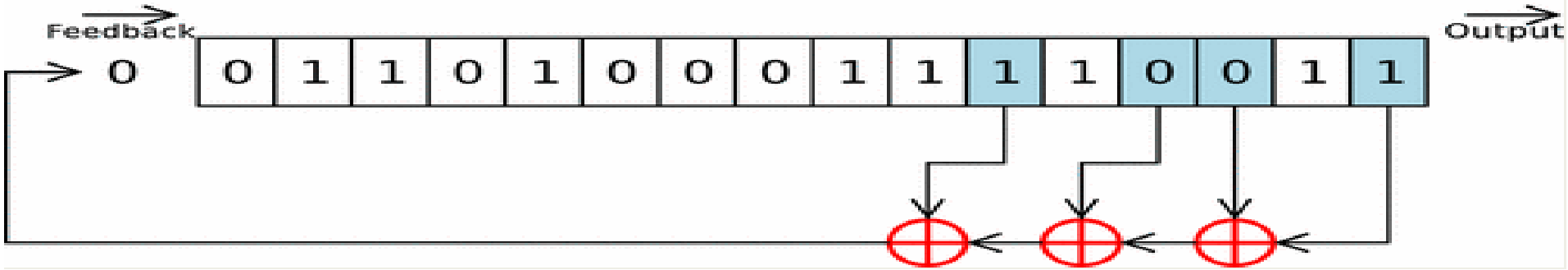
Chiffrement par flot à base de décalage de registre: LFSR

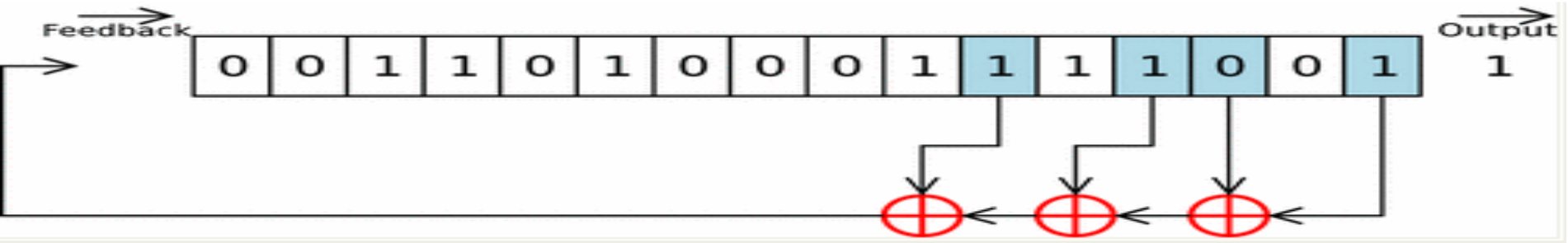


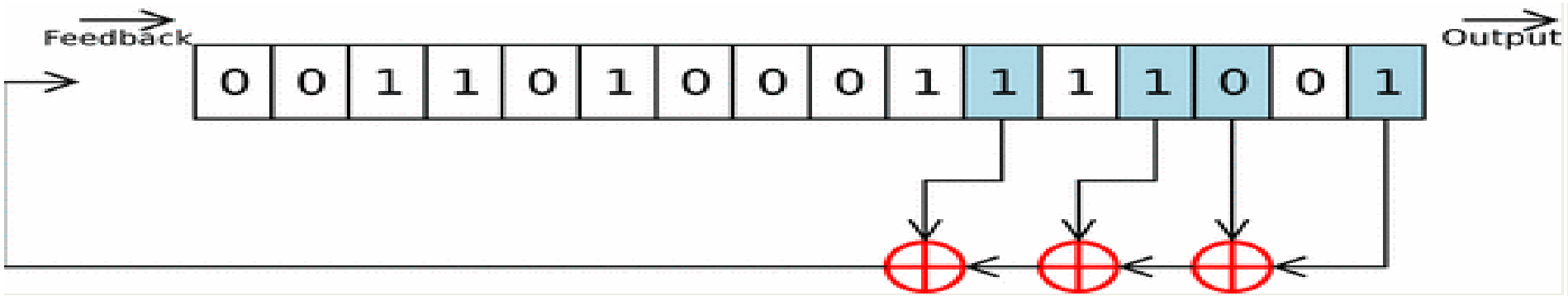




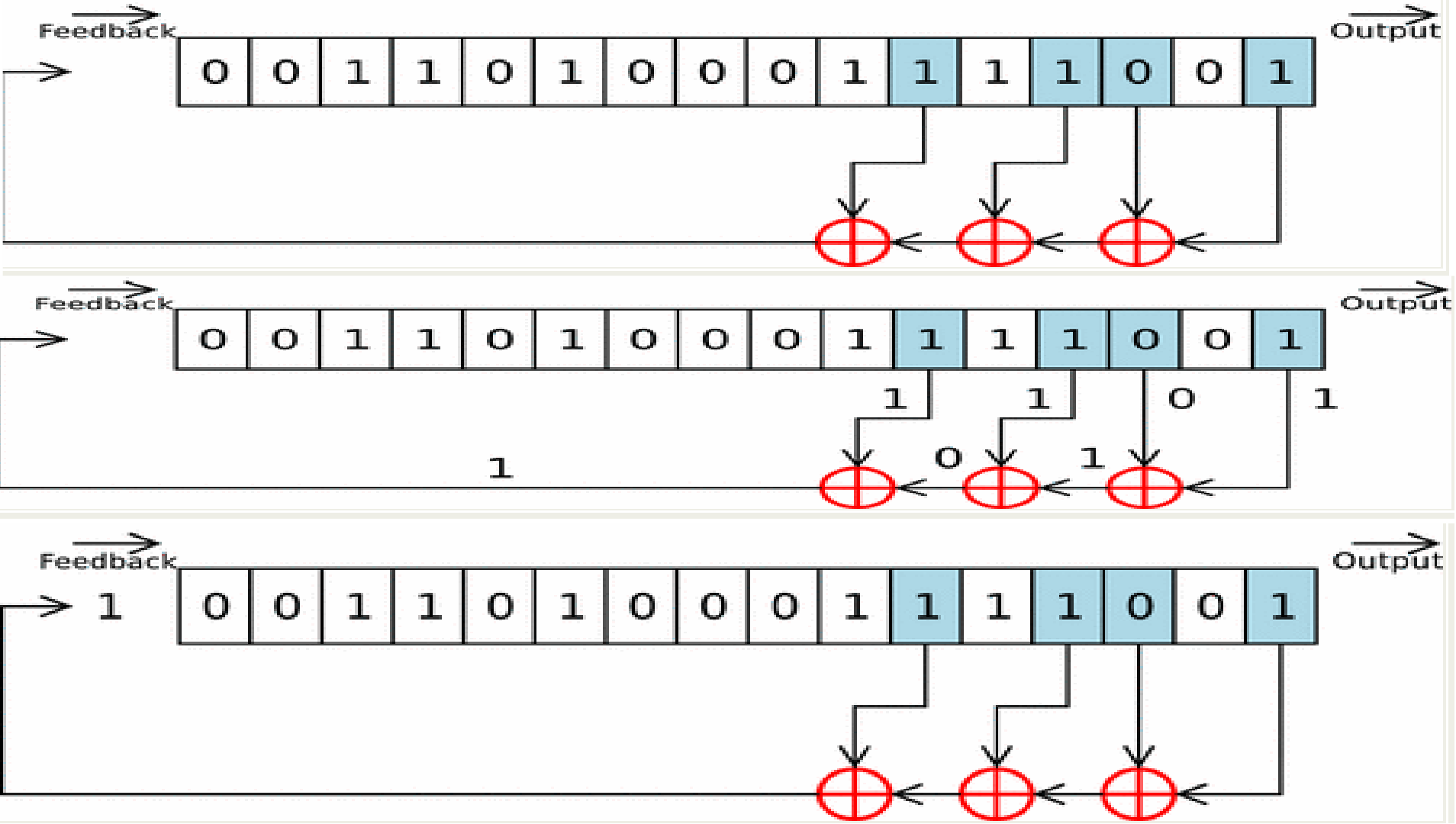
Chiffrement par flot à base de décalage de registre: LFSR



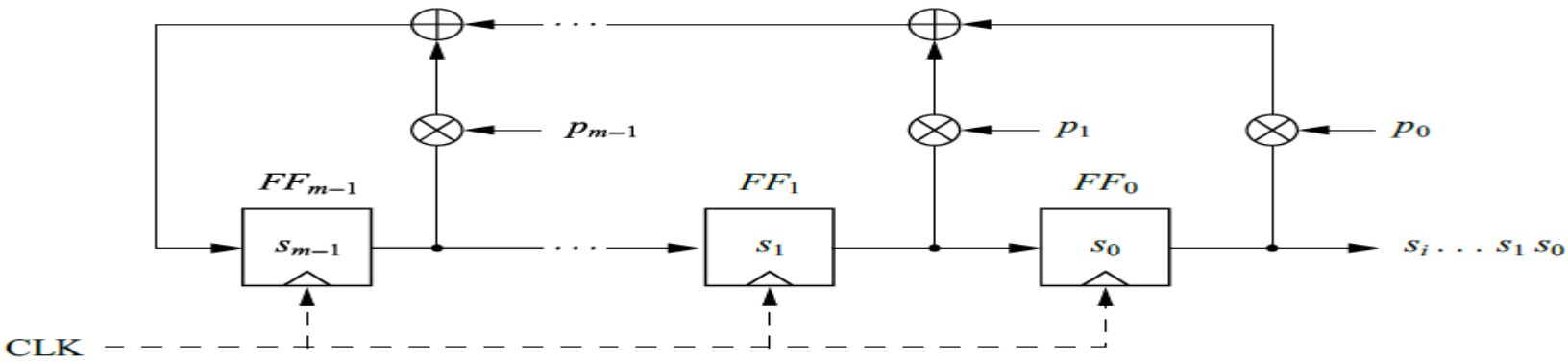




Chiffrement par flot à base de décalage de registre: LFSR

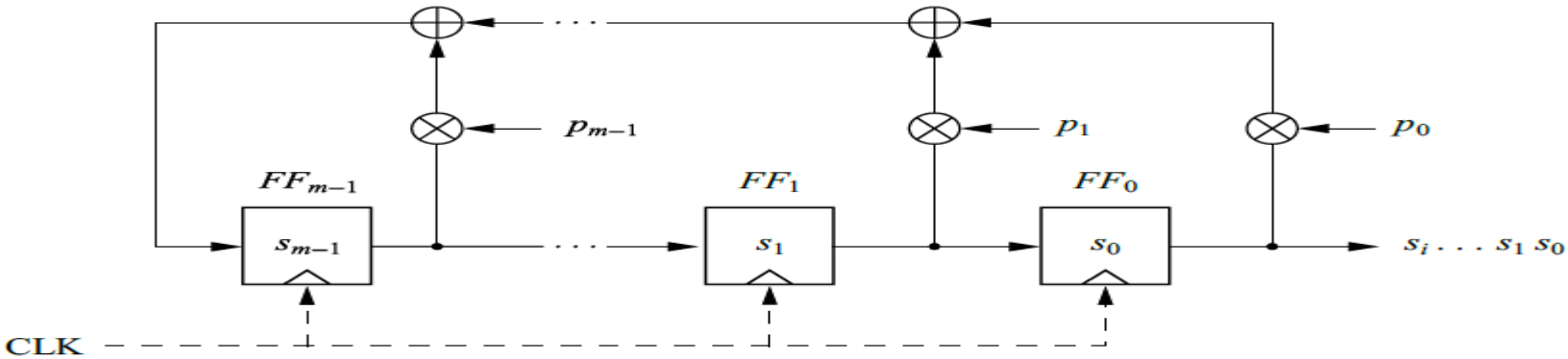


Chiffrement par flot à base de décalage de registre: LFSR



- *The maximum sequence length generated by an LFSR of degree m is* $2^m - 1$.

Known-Plaintext Attack Against Single LFSRs



- *Si on suppose que l'attaquant connait :*

Tous les Yi, le degré m et une partie des Xi i=0 à 2m-1.

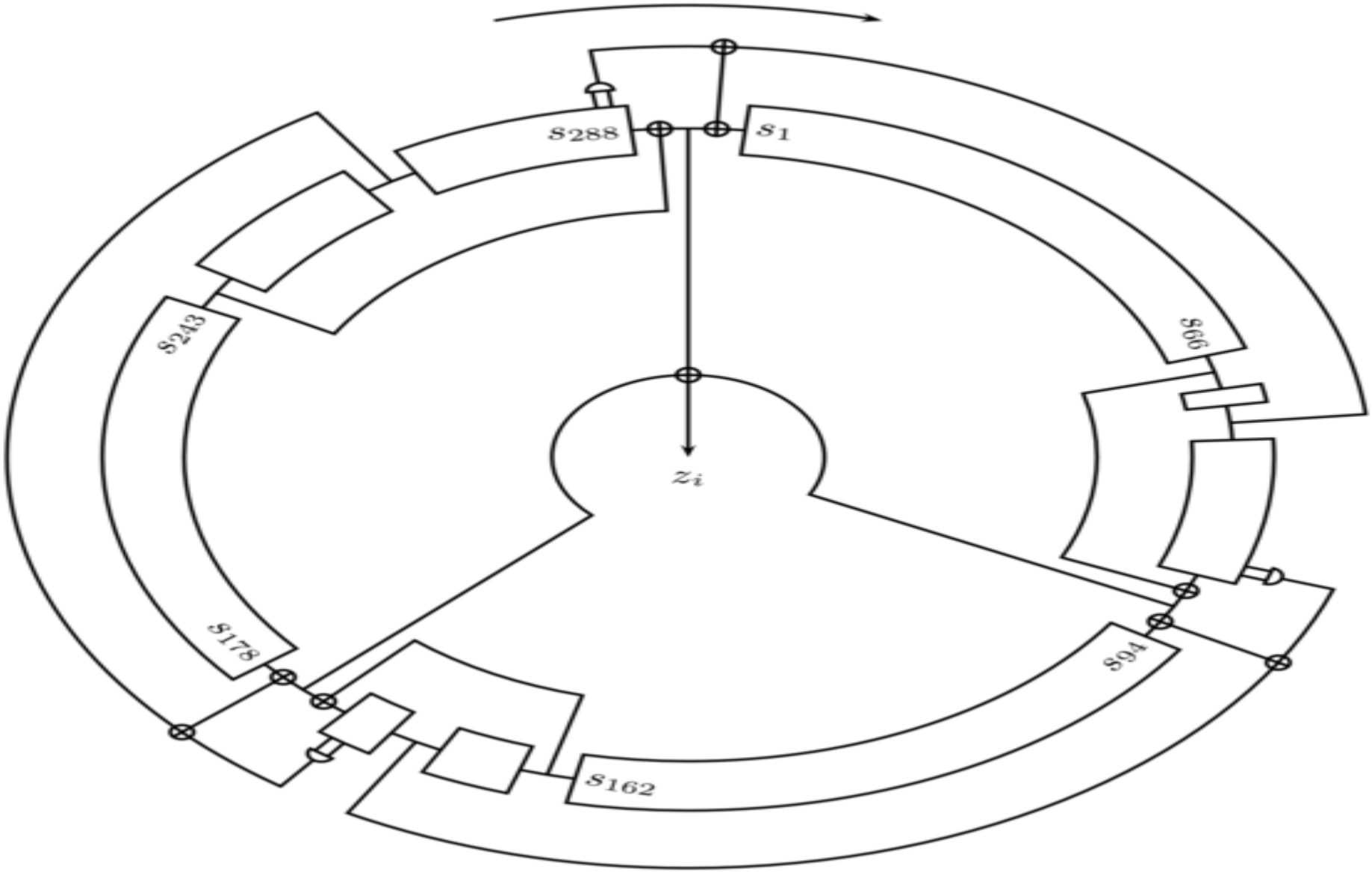
Il peut calculer si ≡ xi+ yi mod 2; i = 0,1, . . . ,2m−1.

- **On a aussi les formules avec coefftion**

$$\left.\begin{array}{l} sm \equiv pm-1sm-1+\ldots+p1s1+p0s0 \bmod 2 \\ sm+1 \equiv pm-1sm+\ldots+p1s2+p0s1 \bmod 2 \\ \ldots \\ s_{2m-1} \equiv pm-1\ s_{2m}-2+\ldots+p1sm+p0sm-1 \bmod 2 \end{array}\right\}$$

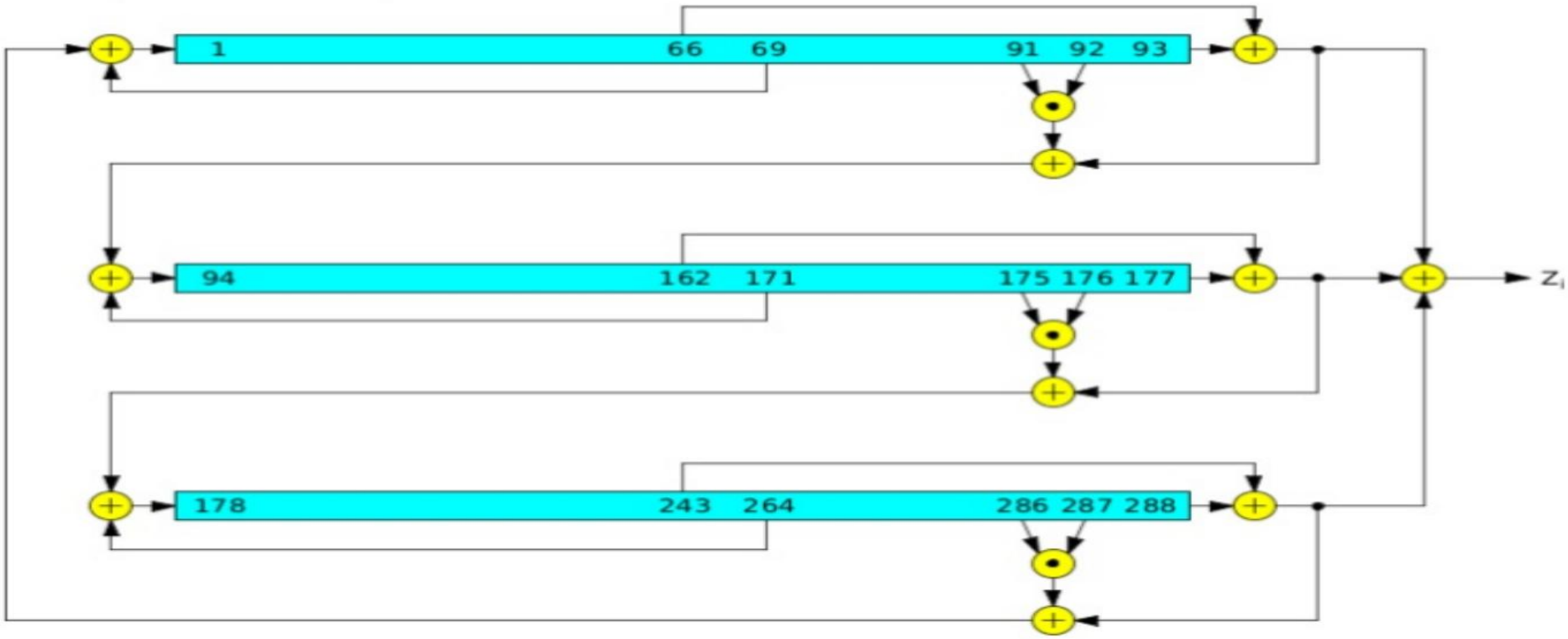
On peut résoudre ça avec gauss elimination method

Chiffrement par flot à base de décalage de registre: Trivium (eSTREAM 2008)



Chiffrement par flot à base de décalage de registre: Trivium (eSTREAM 2008)

3 FSR couplés (de 93, 84, 111 bits)
état = 288 bits
IV : 80 bits
s : 80 bits
init : remplissage
+ 4×288 itérations.



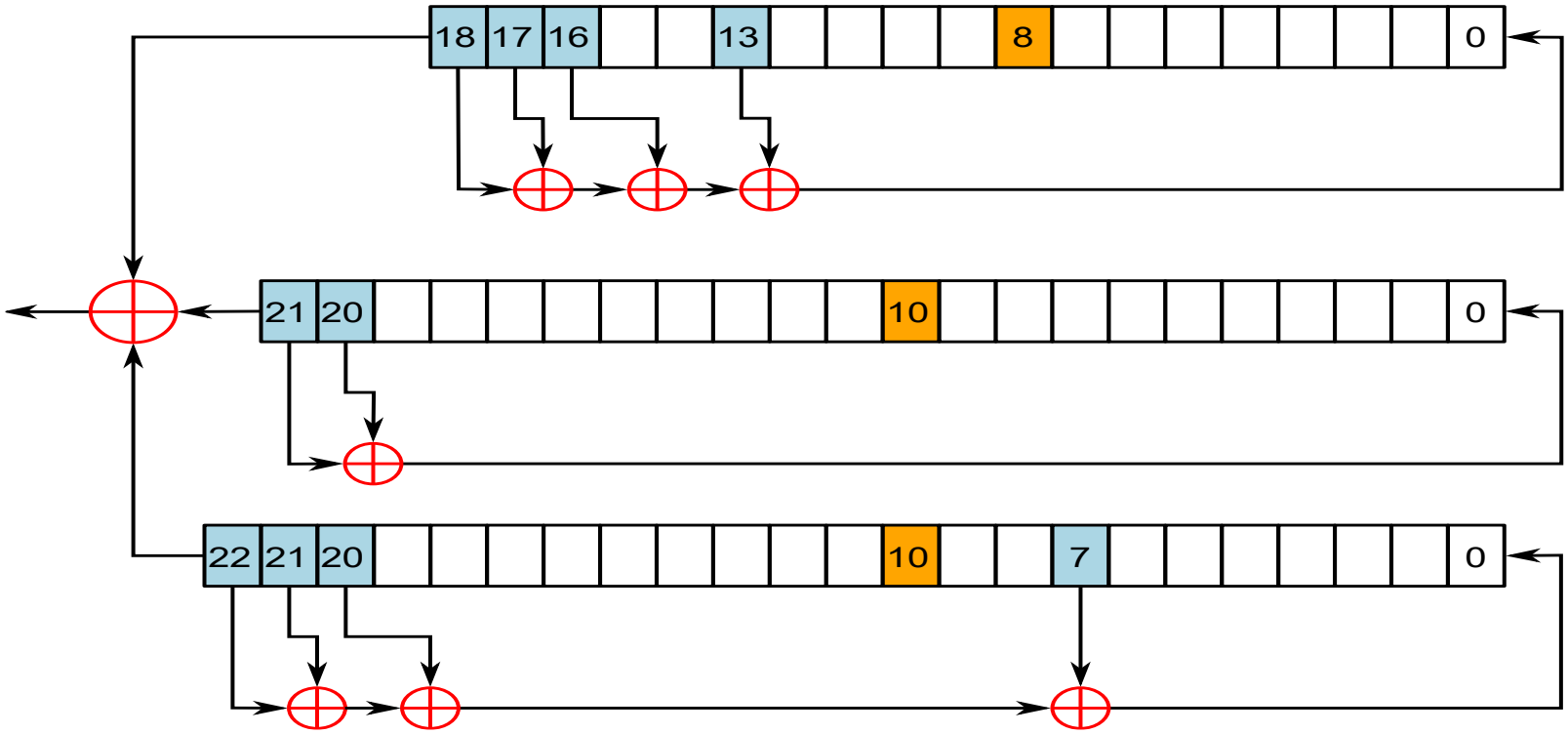
Chiffrement par flot à base de décalage de registre: A5/1 GSM

Les bits chiffrant produit par 3 LFSR de longueur 19, 22 et 23 bits et de polynômes de rétroaction :

$P1(x) = x^{19} + x^{18} + x^{17} + x^{14} + 1;$

$P2(x) = x^{22} + x^{21} + 1;$

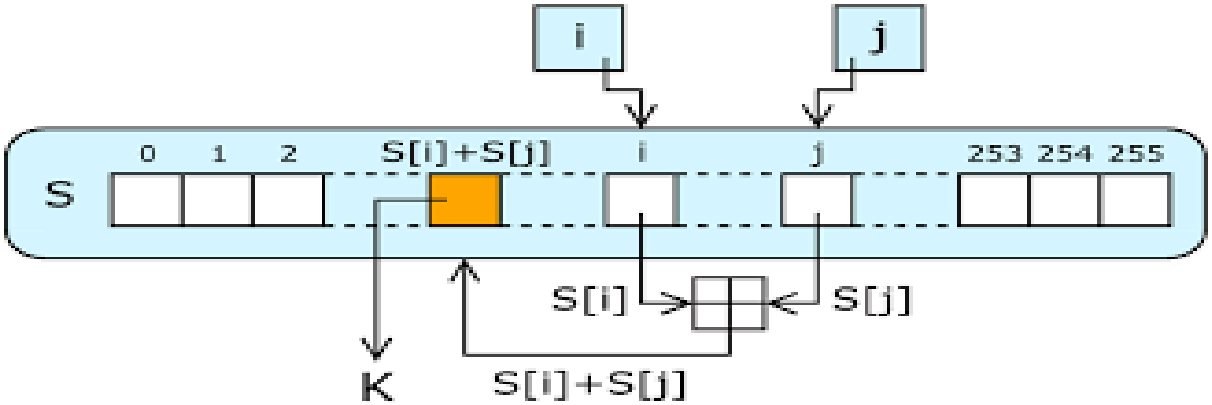
$P3(x) = x^{23} + x^{22} + x^{21} + x^{8} + 1:$



Chiffrement par flot à base de décalage de registre:RC4

RC4 Init

```
Entrée : une clé k de 16 à 256 octets
Sortie : l'état initial (S,i,j)
for i = 0 to 255:
S[i] = i
n = longueur(k)
j = 0
for i = 0 to 255:
j = (j + S[i] + k[i%n])%256
échanger(S[i],S[j])
i = 0
j = 0
return (S,i,j)
```

RC4 GetBits

```
Entrée : l'état courant (S,i,j)
Sortie : un octet y et l'état mis à
jour (S,i,j)
i = (i + 1)%256
j = (j + S[i])%256
échanger(S[i],S[j])
t = (S[i] + S[j])%256
y = S[t]
return (S,i,j), y
```